Multi-UAV Coverage Path Planning Based on Balanced Graph Partitioning

Mengfan Cao, Zaiyue Yang, *Senior Member, IEEE*, and Haoyu Miao

***Abstract*—Coverage path planning is a critical problem in multi-unmanned aerial vehicle (UAV) systems for daily inspection and patrol missions, requiring coordinated movements of UAVs along predetermined trajectories to cover the entire map. Existing methods are often inadequate in considering both workload balance and motion efficiency during task assignment, resulting in imbalanced coverage areas or unnecessary turning maneuvers. To address these limitations, this letter presents a hierarchical structure that integrates spatial decomposition, balanced task assignment through graph partitioning, and turn-aware trajectory planning. The approach follows three key steps: first, spatial decomposition partitions the map into subregions; second, the partitioned map is transformed into a weighted graph, where task assignment distribution is optimized by iteratively migrating and swapping nodes among subgraphs while preserving connectivity, ensuring a balanced workload distribution across UAVs. Finally, trajectory planning incorporates turn optimization to reduce completion time. The results show that our method ensures even task assignment among UAVs, fewer turns, faster task completion, and lower computation time compared to other popular algorithms.**

***Index Terms*—Coverage path planning, graph partitioning.**

I. Introduction

UNMANNED unmanned aerial vehicle (UAV) systems play a crucial role in various applications, including urban infrastructure patrols, environmental monitoring, precision agriculture and disaster assessment. Similar coverage path planning (CPP) challenges also arise in other robotics applications, such as robot vacuum cleaners and autonomous lawn mowers. In both types of applications, the robots are required to efficiently explore large and complex areas, making coverage path planning (CPP) essential to ensure optimal paths and avoid obstacles [1].

Although single-UAV CPP suffices for basic coverage, its scalability is limited by onboard energy and sensor range. Coordinated multi-UAV deployments overcome these limitations by enabling parallel task execution over expansive areas, significantly improving mission completion time and overall efficiency.However, effective multi-UAV cooperation requires well-designed task assignment and trajectory planning strategies to ensure complete coverage while avoiding obstacles and inter-agent collisions. To this end, the CPP problem in multi-UAV scenarios must address three interdependent tasks: 1) decompose the region into subregions without obstacles; 2) assigning the coverage workload to UAVs; 3) optimize the flying trajectory by reducing both the total path length and the number of turns, as frequent turning slows down UAVs and increases the overall mission time.

Previous studies have proposed two main approaches to achieve complete coverage in the CPP problem: cell decomposition methods and grid-based methods. Cell decomposition methods involve partitioning the map of the environment into simple, non-overlapping regions. Depending on the way of cell division, these methods include Trapezoidal, Boustropheon, and Morse-based decomposition [2]. Trapezoidal decomposition partitions the environment into trapezoidal cells by introducing vertical slices on the vertices of the polygon, creating a structured coverage pattern ideal for polygonal environments [3]. Boustrophedon cell decomposition [4] allows the UAV to move back-and-forth along the coverage lines within each cell. Morse-based decomposition uses critical points from Morse functions to create adaptive cell structures [5]. Unlike cell decomposition, grid-based methods partition the area into uniform cells sized by the UAV's coverage range. One common method is the Spanning Tree Coverage (STC) approach [6], where each tree node corresponds to a grid cell. Huang et al. extend STC by constructing the spanning tree using variable-sized grid cells to better handle complex environments [7].

To balance the workload of coverage tasks among UAVs, [8] reformulates the CPP problem as finding the maximum independent set of a bipartite graph and employs a greedy strategy for task assignment, though its computation speed decreases rapidly in large-scale scenarios. In multi-UAV cooperative region scanning, [9] balances tasks by dividing the area among UAVs. In [10], the authors propose a multirobot forest coverage method, which uses a polynomial-time algorithm to achieve efficient coverage by balancing tree weights. [11] employs an offline coverage spanning tree method combined with an online coverage path planning approach. A classical approach for task allocation involves converting the task assignment problem into a Traveling Salesman Problem (TSP). In [12], clustering method with vehicle routing is integrated to complete task. [13] employs a k-means clustering method is to convert the task assignment problem into a TSP and solves it using heuristic methods. [14] formulates task allocation as a minimum

Received 8 October 2025; accepted 2 February 2026. Date of publication 16 February 2026; date of current version 26 February 2026. This article was recommended for publication by Associate Editor R. Gross and Editor M. A. Hsieh upon evaluation of the reviewers' comments. The work was supported by Guangdong Science and Technology Program under Grant 2024B1212010002. *(Corresponding author: Zaiyue Yang.)*

The authors are with the Guangdong Provincial Key Laboratory of Fully Actuated System Control Theory and Technology, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: 12331364@mail.sustech.edu.cn; yangzy3@sustech.edu.cn; haoyumiao97@gmail.com).

Digital Object Identifier 10.1109/LRA.2026.3665069

Hamiltonian partition problem and solves it using heuristic partitioning with multi-TSP routing. Similarly, [15] employs edge-based heuristics to solve the k-TSP problem for task allocation.

To optimize UAV trajectories, [16] transforms the area coverage problem into a line coverage problem and employs the Merge-Embed-Merge algorithm to generate coverage routes. [17] formulates the coverage path within each pre-assigned region as a TSP and solves it using dynamic programming. [18] models the task assignment and routing as a multiple TSP (mTSP) and proposes a branch-and-bound approach to find near-optimal tours. To further reduce the number of turns, [19] employs a heuristic rank decomposition to minimize turns and formulates a mTSP on the ranks to optimize mission time. [20] proposes an iterative method to optimize partitioning lines and formulates a generalized TSP to minimize the number of turns. However, its computation time grows rapidly with problem size due to the NP-hard nature of TSP.

Despite recent progress in CPP, current research mainly focuses on task assignment based on path length, with less consideration given to the cost of turns. Jointly minimizing both path length and the number of turns remains a computationally intractable NP-hard problem [21]. Furthermore, task assignment and trajectory optimization are inherently coupled with each other, complicating their joint optimization. To address these limitations, we transform the map into a graph and propose a balanced graph partitioning approach to achieve optimal task assignment. Moreover, we plan trajectories with fewer number of turns to minimize the overall completion time. Our contributions are summarized below.

1) For the multi-UAV CPP problem, we design a hierarchical structure and perform graph partitioning considering node weights, where node weights represent the completion time of each subregion. The balanced graph partitioning approach enables efficient computation, ensuring balanced task load, and reducing overall task completion time.
2) For each subregion, we first determine the optimal coverage pattern based on its shape. Then we utilize the spanning tree structure to construct a connected route with minimal number of turns.
3) We evaluate our algorithm on maps with varying obstacle densities, comparing it with other popular approaches. The results show that our method outperforms these approaches in reducing the number of turns and minimizing the completion time, while still preserving fast computation.

The remainder of this letter is organized as follows. Section II introduces a graph-based region partitioning framework. Section III addresses multi-UAV task assignment via node-weighted graph partitioning. Section IV develops a turn-minimizing trajectory planning method based on spanning tree optimization. Results are presented in Section V, followed by conclusions in Section VI.

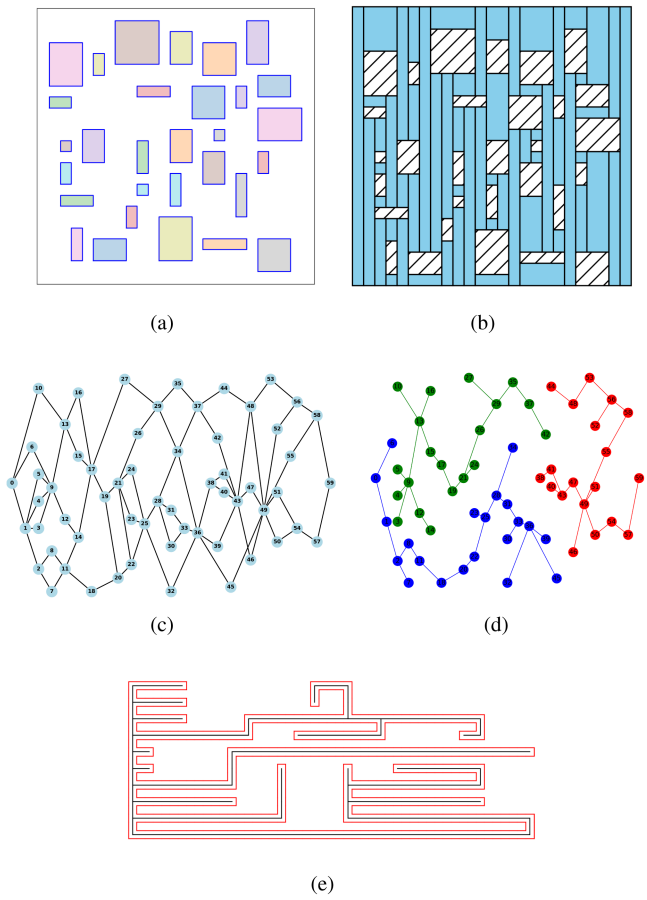

Fig. 1. Task assignment based on topology graph partitioning: (a) Random map (b) Decomposed coverage region (c) Topological structure diagram (d) Task assignment (e) Trajectory path.

II. Problem Description

To address the multi-UAV CPP problem with collision avoidance, we decompose complex environments into obstacle-free subregions and assign tasks to each UAV through coordinated planning. The process is structured into four key steps, as shown in Fig. 1.

i) Region decomposition (a)-(b): The entire feasible area is decomposed into interconnected, obstacle-free rectangular subregions suitable for UAV coverage.
ii) Graph construction (b)-(c): Subregions are mapped to nodes with weights representing coverage time. Edges are defined by spatial distances between subregions.
iii) Task assignment (c)-(d): The graph is partitioned into tree-structured subgraphs with balanced total node weights, ensuring even task distribution across UAV groups.
iv) Trajectory generation (d)-(e): For each subgraph, a closed-loop, collision-free trajectory is generated, minimizing turns across the assigned subregions.

To proceed further, the following assumptions are considered for UAVs; the sensor carried by the UAV has a field of view of $r \times r$, where r is a positive constant; the altitude at which each UAV flies is constant, then CPP problem can be formulated in

a 2-dimensional space; each UAV flies at a speed α_v except for the turning phase, where it turns at an angular velocity α_w.

A. Region Decomposition

As shown in Fig. 1(a), the colored rectangles inside the target region are obstacles or no-fly zones. To avoid collisions with obstacles, the feasible region is decomposed by an exact cell decomposition method.

To efficiently partition the region, we adopt the sweep line method [22], which has a low computational complexity of $O(n \log n)$, and produces convex trapezoidal cells. Subregions with an area exceeding twice the average are partitioned perpendicular to the shorter edge L^s of their Minimum-Area Enclosing Rectangle [23] to minimize coverage turns. The number of subdivisions m is defined as:

$$m = \min\left\{\left\lfloor \frac{L^s}{2r} \right\rfloor,\ k\right\}. \tag{1}$$

This bound ensures physical feasibility while capping partitions at the UAV count k to avoid unnecessary graph complexity. Regarding complexity, this subdivision process is linear $O(m)$ for rectangular regions, while trapezoidal regions require $O(m \log m)$ due to the binary-search-based alignment. UAVs are then assigned to non-overlapping feasible subregions, as illustrated in Fig. 1(b), where the blue areas represent feasible subregions.

B. Graph Construction

Following region decomposition, the environment is modeled as a weighted graph $G(V, E, W)$, where each node $v_i \in V$ represents a subregion centered at its geometric center. In this graph, node and edge weights characterize coverage workloads and spatial distances, respectively.

1) Edge Weight: The set E contains all edges, where an edge e_{ij} connects nodes v_i and v_j if they are adjacent. The edge weight d_{ij} represents the Euclidean distance between nodes v_i and v_j, calculated as:

$$d_{ij} = \|v_i - v_j\|_2. \tag{2}$$

2) Node Weight: W is node weight, and $\omega(v_i)$ denotes the weight of subregion i, which corresponds to the time required to complete the coverage of this subregion.

For a rectangular subregion of size $l_i^l \times l_i^s$, the UAV can sweep either vertically (along columns, Fig. 2(a)) or horizontally (along rows, Fig. 2(b)) in a back-and-forth lawnmower pattern within $r \times r$ grid cells (orange lines), centered at red dots representing $2r \times 2r$ cells. Vertical sweeping of a 6×4 subregion (Fig. 2(a)) covers 24 cells with 8 turns, while horizontal sweeping (Fig. 2(b)) covers the same area but requires 12 turns. This demonstrates how the sweep direction affects both the number of turns and the completion time.

The total completion time is determined by the total path length and the number of turns incurred during directional changes. For horizontal sweeping, the UAV moves back and forth across a distance of l_i^s, repeating this motion $\lceil l_i^l/2r \rceil$ times, where $\lceil \cdot \rceil$ denotes rounding up to the nearest integer. The path

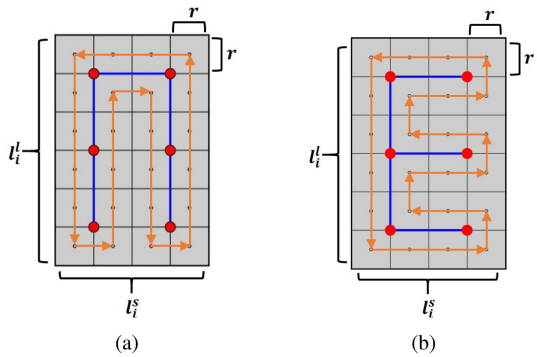

Fig. 2. Two scanning directions: (a) Sweep vertically, and (b) Sweep horizontally.

traversal time is then $\frac{2\lceil l_i^s \cdot \lceil l_i^l/2r \rceil}{\alpha_v}$, where α_v is the UAV's linear velocity. Similarly, for vertical sweeping, the path traversal time is $\frac{2\lceil l_i^l \cdot \lceil l_i^s/2r \rceil}{\alpha_v}$. The turn time is $\frac{\pi}{2\alpha_w}$, with $4\lceil l_i^l/2r \rceil$ and $4\lceil l_i^s/2r \rceil$ turns for horizontal and vertical sweeping, respectively.

Thus, the node weight $\omega(v_i)$ is defined as the minimum total completion time of the two sweeping strategies:

$$\omega(v_i) = \min\left\{\frac{2l_i^s \cdot \lceil l_i^l/2r \rceil}{\alpha_v} + \frac{2\pi\lceil l_i^l/2r \rceil}{\alpha_w},\ \frac{2l_i^l \cdot \lceil l_i^s/2r \rceil}{\alpha_v} + \frac{2\pi\lceil l_i^s/2r \rceil}{\alpha_w}\right\} \tag{3}$$

III. Balanced Workload Via Graph Partitioning

Building upon the graph representation $G(V, E, W)$, we formulate the multi-UAV task assignment as a graph-based optimization problem. As shown in the transformation from Fig. 1(c) to (d), the goal is to partition the graph into K connected, obstacle-free subgraphs with tree structures, ensuring balanced workloads among UAVs. Here, node weights represent the completion time required for each subregion, while edge weights are used solely to capture the physical position.

Denote subgraph $S_i = (V_i, E_i)$, where $V_i \subseteq V$ is the subset of nodes, and $E_i \subseteq E$ is the subset of edges, i.e., $E_i = \{(v_j, v_k) \in E \mid v_j, v_k \in V_i\}$. The total weight $W(S_i)$ of a subgraph S_i is the sum of the weights of the nodes it contains, i.e.,

$$W(S_i) = \sum_{v \in S_i} w(v). \tag{4}$$

The goal is to minimize the total difference between the weight of each subgraph and the average total weight, while ensuring that the subgraphs remain connected. Thus, the optimization problem can be stated as follows,

$$\begin{aligned} \min \quad & \sum_{i=1}^{k}\Big(W(S_i) - \tfrac{1}{k}\sum_{i=1}^{k} W(S_i)\Big)^2 \\ \text{s.t.} \quad & \bigcup_{i=1}^{k} V_i = V, \\ & S_i \cap S_\ell = \emptyset, \quad \forall i \neq \ell, \end{aligned}$$

$$S_i \text{ is connected.} \tag{5}$$

This problem is NP-hard in general, as it combines the challenges of both balanced partitioning and connectivity constraints. An exact solution is computationally expensive for large-scale graphs.

To address this challenge efficiently, a two-step heuristic approach is proposed: subgraph initialization and subgraph balancing via node migration and node swap. We first construct an initial partition by selecting initial nodes via weighted k-means clustering. The initial graph partitioning is then refined by iteratively transferring nodes between neighboring subgraphs to balance the total node weights, while maintaining connectivity.

A. Subgraph Initialization

To ensure a stable starting configuration that accounts for workload heterogeneity, the initial nodes are selected to represent different regions of the graph based on the distribution of node weights. This is achieved via weighted k-means clustering, where each cluster centroid c_i^* is computed using the weighted positions of the nodes. The initial node is denoted as follows,

$$v_i^{(0)} = \arg\min_{v_j \in S_i} \|v_j - c_i^*\|, \tag{6}$$

which $\{c_i^*\}_{i=1}^k \subset \mathbb{R}^d$ denotes the centroids of the clusters.

At each iteration, each subgraph $\mathcal{S}_j$ identifies its candidate set Γ_j of unassigned neighboring nodes. From candidate set, node u^* is selected such that the updated subgraph weight $W_j + w(u^*)$ is closest to the average subgraph weight $\bar{W}$. This selection is formalized as,

$$u^* = \arg\min_{u \in \Gamma_j} \left| W_j + w(u) - \bar{W} \right|, \tag{7}$$

where $W_j = \sum_{v \in V_j} w(v)$ is the total weight of subgraph $\mathcal{S}_j$, and $\bar{W} = \frac{1}{k} \sum_{v \in V} w(v)$ is the average subgraph weight. Each selected node is added to the corresponding subgraph along with the connecting edge to maintain connectivity. Algorithm 1 summarizes above procedures.

B. Subgraph Balancing Via Node Migration

After the initial graph construction is completed, we design a balancing heuristic that integrates both node migration and node swap operations to further improve weight balance. Fig. 3 illustrates main steps of subgraph balancing process. It begins with two initial subgraphs, as shown in Fig. 3(a). To reduce the imbalance, node migration is first attempted, as illustrated in Fig. 3(b). If migration cannot further improve balance, node swapping between subgraphs can be applied, as shown in Fig. 3(c). Finally, Fig. 3(d) shows the subgraphs after balancing operations.

Denote $\phi : V \to \{1, \ldots, k\}$ as the mapping that assigns each node to a subgraph. The node v belongs to subgraph S_i if and only if $\phi(v) = i$. For each subgraph S_i, its boundary edge set B_i is defined as the set of edges connecting a node inside S_i to a node in other subgraphs,

$$B_i = \{(u, v) \in E \mid \phi(u) = i \wedge \phi(v) \neq i\}. \tag{8}$$

Algorithm 1: Subgraph Initialization.

Input: the graph $G = (V, E)$, number of subgraphs$k$, initial seed nodesS_0
Output: $\mathcal{S}$: list of subgraphs, $\mathcal{W}$: subgraph weights

Initialize: ;
for $j \leftarrow 1$ **to** k **do**
  $\mathcal{S}[j] \leftarrow \text{Graph}(\{S_0[j]\}, \emptyset)$;
  $\mathcal{W}[j] \leftarrow w(S_0[j])$; $\phi[S_0[j]] \leftarrow j$;
Expansion Phase: while $|\phi| < |V|$ **do**
  for $j \leftarrow 1$ **to** k **do**
    Denote boundary edge set B_j;
    if $\Gamma_j \neq \emptyset$ **then**
      select the node u^* based on eq.(7);
      Update node set $\mathcal{S}[j]$;
      Update the workload weight $\mathcal{W}[j]$;
      Update the subgraph label $\phi[u^*] \leftarrow j$;
      for $(u^*, v) \in E$ ***and*** $v \in N_j$ **do**
        $E[j] \leftarrow E[j] \cup \{(u^*, v)\}$;
return $\mathcal{S}, \mathcal{W}$

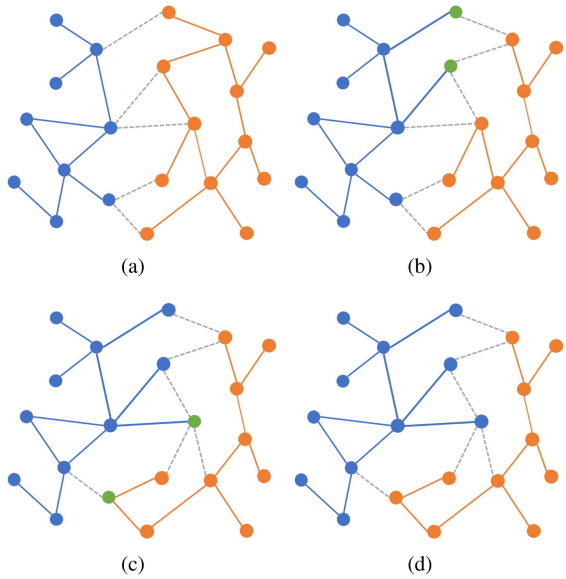

Fig. 3. (a) Subgraphs (b) Node migration (green nodes indicate migrated nodes) (c) Node swapping (green nodes indicate swapped nodes) (d) Subgraphs after iteration.

Since S_i may be adjacent to multiple subgraphs, the boundary edge set captures all potential node movements or exchanges.

To handle weight imbalance, a migration process is triggered whenever the weight of any subgraph exceeds the average weight by a margin ε,

$$W(S_i) > \frac{1}{k} \sum_{j=1}^{k} W(S_j) + \varepsilon, \tag{9}$$

where ε is set to the minimum node weight of the graph, reducing it further does not improve balance.

The candidate edge set $\mathcal{E}_{\text{cand}}$ is identified as boundary edges that maximize the weight difference between the connected subgraphs,

$$\mathcal{E}_{\text{cand}} = \arg\max_{(u,v)\in B_i} \left|W(S_{\phi(u)}) - W(S_{\phi(v)})\right|. \tag{10}$$

From these candidate edges, a subset of migration pairs $\mathcal{M}$ is selected to jointly minimize the imbalance. Nodes in the overloaded subgraph S_i are migrated simultaneously to neighboring subgraphs, if connectivity is preserved,

$$\begin{aligned}\mathcal{M}^* = \quad & \arg\min_{\substack{\mathcal{M}\subseteq\mathcal{E}_{\text{cand}} \\ \phi(u)=i,\phi(v)=j}} \left(W(S_i) - W(S_j) - 2\sum_{(u,v)\in\mathcal{M}} w(u)\right)^2 \\ s.t. \quad & S_i \setminus \{u \mid (u,v)\in\mathcal{M}\} \text{ is connected}, \\ & S_j \cup \{u \mid (u,v)\in\mathcal{M}\} \text{ is connected}. \end{aligned} \tag{11}$$

C. Subgraph Balancing Via Node Swap

If node migration cannot reduce the objective or breaks connectivity, a node swap strategy between S_i and S_j is attempted to improve balance, which allows one or more nodes from S_i to swap with one or more nodes from S_j, provided connectivity is preserved. Formally, for candidate sets $\mathcal{U}\subseteq S_i$ and $\mathcal{V}\subseteq S_j$, the optimal swap is

$$\begin{aligned}\mathcal{M}^* = \arg\min_{\substack{\mathcal{U}\subseteq S_i,\,\mathcal{V}\subseteq S_j \\ (\mathcal{U},\mathcal{V})\neq\emptyset}} & \Big|(W(S_i) - W(S_j)) \\ & - 2\left(\sum_{u\in\mathcal{U}} w(u) - \sum_{v\in\mathcal{V}} w(v)\right)\Big| \\ s.t. \quad & (S_i \setminus \mathcal{U}) \cup \mathcal{V} \text{ is connected}, \\ & (S_j \setminus \mathcal{V}) \cup \mathcal{U} \text{ is connected}. \end{aligned} \tag{12}$$

The balancing process employs node migration first, and applies swap operations if migration fails to improve the balance. To ensure computational efficiency, the search for swap candidates is restricted to boundary nodes and their immediate neighbors. Furthermore, connectivity is verified in constant time via Tarjan's algorithm. Consequently, the complexity of Algorithm 2 is $O(I\cdot|B|^2)$ over I iterations, covering both collective node migration and swap operations. Since candidate selection is strictly limited to the boundary B and its local neighborhood, this strategy maintains high scalability for large-scale environments where $|B| \ll |V|$. The procedure terminates in a finite number of steps as workload discrepancy decreases monotonically until convergence, as stated below.

Lemma 1: Algorithm 2 converges in a finite number of iterations.

Proof: Define the imbalance objective as

$$\mathcal{J}(\mathcal{S}) = \sum_{i=1}^{k} \left(W(S_i) - \bar{W}\right)^2, \tag{13}$$

where $\bar{W}$ is the average subgraph weight. Clearly $\mathcal{J}(\mathcal{S}) \geq 0$.

The algorithm enforces the non-increasing imbalance condition

$$\left|W(S_i) - W(S_j) - 2w(\mathcal{M}^*)\right| < |W(S_i) - W(S_j)|, \tag{14}$$

Algorithm 2: Subgraph Balancing via Node Migration and Swap.

Input: $G=(V,E)$: the graph, k: number of subgraphs, $\mathcal{S}$: initial subgraph partition, ϵ: weight imbalance threshold, max_iter: maximum iterations

Output: $\mathcal{S}$: adjusted subgraph partition

```
Initialize: for i ← 1 to k do
    Compute initial subgraph weight;
Main Loop: while iteration ≤ max_iter do
    for i ← 1 to k do
        if |W(S_i) − (1/k) Σ_{j=1}^{k} W(S_j)| > ε then
            for M ∈ E_cand do
                Select candidate by Eq. (10);
                Migration nodes and update node set
                  S_i, S_j by Eq.(11);
                if no migration applied then
                    select U ⊆ S_i, V ⊆ S_j;
                    Swap U, V and update S_i, S_j by
                      Eq. (12);
            if S = S_prev then
                S ← MST(S_i);
                return S
            else
                iteration ← iteration + 1;
```

which ensures that the imbalance strictly decreases. For a migration of nodes $\mathcal{M}^*$ with total weight $w(\mathcal{M}^*)$ from S_i to S_j, only S_i and S_j are affected, and the change in $\mathcal{J}$ is

$$\Delta\mathcal{J} = 2w(\mathcal{M}^*)[w(\mathcal{M}^*) + W(S_j) - W(S_i)], \tag{15}$$

which is always negative under the above condition. Hence, every successful migration strictly reduces $\mathcal{J}$.

If no migration can further reduce J, the algorithm attempts a node swap between S_i and S_j. Let $\mathcal{U}\subseteq S_i$ and $\mathcal{V}\subseteq S_j$ be the sets of nodes to exchange, with total weights $w(\mathcal{U}) = \sum_{u\in\mathcal{U}} w(u)$ and $w(\mathcal{V}) = \sum_{v\in\mathcal{V}} w(v)$. The swap is accepted only if

$$\left|W(S_i) - W(S_j) - 2(w(\mathcal{U}) - w(\mathcal{V}))\right| < \left|W(S_i) - W(S_j)\right|, \tag{16}$$

which again ensures $\Delta\mathcal{J} < 0$.

After the swap, the subgraph weights are update as

$$\begin{aligned} W(S_i) &\leftarrow W(S_i) - w(\mathcal{U}) + w(\mathcal{V}), \\ W(S_j) &\leftarrow W(S_j) - w(\mathcal{V}) + w(\mathcal{U}). \end{aligned} \tag{17}$$

From the convergence analysis, since $\Delta\mathcal{J} < 0$, every migration or swap operation moves $\mathcal{J}$ closer to the optimum.

IV. Branch Merging and Trajectory Generation

Following task assignment, UAV coverage trajectories are generated. For each assigned area (Fig. 4(a)), the corresponding

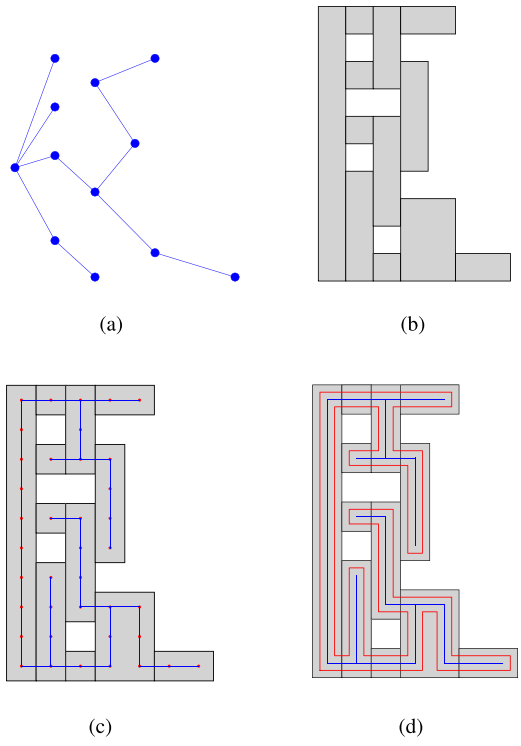

Fig. 4. (a) Subgraphs (b) Subregions (c) Branch merging with tree construction (d) Path generation.

subregions are extracted (Fig. 4(b)) and decomposed into parallel scanning branches according to (3). The adjacent scanning branches are then linked by connecting their endpoints, ensuring path continuity. This merging process forms a spanning tree, as shown in Fig. 4(c). Finally, the tree is closed into a loop to form a complete and efficient trajectory, as depicted in Fig. 4(d). This process ensures that UAV performs regional coverage with minimized turns and improved overall efficiency.

Algorithm 3 constructs a spanning tree T by merging subgraph edge sets F_i. For each subregion, T_i is built by connecting edges with common coordinates (horizontal/vertical) to minimize turns. These individual trees are then combined into a globally connected structure T, which is then traversed in $O(|V|)$ to form the trajectory. Overall, the framework operates in polynomial time, specifically $O(n \log n + I|B|^2 + |V|)$, ensuring high scalability for large-scale environments.

V. Simulation Results and Analysis

This section evaluates the proposed multi-UAV coverage algorithm through simulations on randomly generated regions with different obstacle sizes, which are run using Python 3.9 on a Core i7 3.4 GHz CPU with 8 GB RAM. Simulation parameters are as follows: the coverage area per UAV is set to $1m \times 1m$,and the total region ranges from $200m \times 200m$ to $400m \times 400m$, with different types of obstacles. The UAV travel velocity is fixed at $1\,\mathrm{m/s}$, and its rotational velocity at $0.5\,\mathrm{rad/s}$.

We conduct simulations to evaluate the proposed method and compare it with popular approaches, including optimization-based methods [17], solved using by Gurobi, with enhancements

Algorithm 3: Branch Merging for Tree Structure Construction.

Input : $\mathcal{E}$: branches set of Subgraphs
Output: T: tree structure
for $i = 1$ **to** k **do**
  $T_i := \varnothing$;
  while $\exists\, e_{next} \in \mathcal{E}_i$, *and* $e_{next} \notin$ *visited* **do**
    if $e_{next} \sim e$ *(adjacent edges)* ***and*** $e_{next} \notin$ *visited* **then**
      if $x(e_{next}) = x(e)$ *or* $y(e_{next}) = y(e)$ **then**
        $e_{\text{add}} = (v_{\text{end}}(e), v_{\text{start}}(e_{\text{next}}))$;
        $T_i := T_i \cup e_{\text{add}}$;
        Update the visited branches;
        update the current branch;
  $i \leftarrow i + 1$;
return $T = \bigcup_{i=1}^{k} T_i$;

TABLE I
Computational Time Comparison (s)

		Gurobi	LSCPP	DARP	TMSTC*	ACMC	Proposed
case 1	k=3	11.07	84.56	4.42	5.15	3.67	1.95
	k=4	39.24	69.03	4.32	5.25	3.68	1.62
	k=5	33.96	69.44	4.55	5.44	3.71	1.88
	k=6	585.00	68.89	6.66	5.54	3.70	1.59
case 2	k=3	105.01	220.05	6.10	20.21	16.90	4.01
	k=4	455.47	171.85	5.76	20.15	17.60	3.35
	k=5	369.94	145.88	5.77	19.29	16.90	3.32
	k=6	735.42	122.69	6.04	19.55	16.96	2.71
case 3	k=3	43.02	313.23	17.27	84.19	34.08	11.51
	k=4	65.07	392.59	17.14	82.31	34.87	9.74
	k=5	89.94	390.01	19.16	84.81	34.24	7.51
	k=6	165.67	384.63	26.53	85.89	34.18	6.46
case 4	k=3	81.42	542.24	80.88	331.13	124.64	33.87
	k=4	68.38	522.99	159.56	332.10	123.59	30.54
	k=5	185.39	506.94	123.72	332.11	124.20	26.15
	k=6	398.88	599.57	127.37	330.09	124.04	22.55

to the objective function and constraints, and an optimality gap set to 8% . Large-Scale Multi-Robot Coverage Path Planning(LSCPP) [24], Divide Areas based on Robot's Positions (DARP) [9], Turn-Multi-robot Spanning Tree Coverage Star (TMSTC*) [8], and Area Coverage with Multiple Capacity-Constrained Robots (ACMC) [16]. The comparison will focus on task load balance, number of turns, and computation time.

A. Computation Time Comparisons

A statistical analysis of computation times across different scenarios is presented in Table I. In cases 1 and 2, the proposed method, along with DARP and TMSTC*, delivers faster computation than Gurobi and LSCPP, with the proposed method remaining the fastest overall. As the problem scale increases, Gurobi-based optimization becomes limited by problem size and suffers from slow solution times, while LSCPP, DARP and TMSTC* show much longer run times, particularly TMSTC*,

TABLE II
COMPLETION TIME (S)

Case	Method	k=3	k=4	k=5	k=6
case 1	MILP	7042	5394	4670	3758
	LSCPP	7840	6562	6930	5180
	DARP	7136	5500	4372	3808
	TMSTC*	6824	5206	4170	3497
	ACMC	10291.73	6808.29	5190.17	4184.73
	Proposed	6850	5394	4154	3506
	Subgraph Init.	7562	6526	5790	4796
	Migration-only	7036	5596	4638	3572
case 2	MILP	12808	9744	8306	6798
	LSCPP	15820	13022	12372	8976
	DARP	12774	9576	7686	6340
	TMSTC*	12106	9005	7216	6006
	ACMC	18334.60	12729	9519.72	7607.45
	Proposed	12128	9078	7270	6062
	Subgraph Init.	14370	10982	8140	8550
	Migration-only	12150	9100	7300	6094
case 3	MILP	27000	21238	17548	13740
	LSCPP	29417.41	25212.82	24537.41	21443.22
	DARP	27540	20468	16286	14046
	TMSTC*	25573	19241	15445	12845
	ACMC	35642.80	25809.10	19427.70	15540.60
	Proposed	25608	19264	15660	13420
	Subgraph Init.	33320	22488	21736	19216
	Migration-only	26392	19328	15680	13898
case 4	MILP	52144	38328	32760	27296
	LSCPP	51792	38844	33672	27896
	DARP	51710	39544	31482	26398
	TMSTC*	49437	37284	29758	24811
	ACMC	74544.70	49481.30	37395.20	29870.80
	Proposed	49548	37604	29776	25180
	Subgraph Init.	62560	45288	48814	38120
	Migration-only	50276	37900	30004	26558

which is burdened by the high dimensionality of its fine-grained grid representation. In contrast, the advantage of the proposed method becomes more prominent, running approximately 2–5 times faster than DARP and ACMC, and over 10 times faster than TMSTC* in the largest scenarios. These results demonstrate its capability for real-time and large-scale multi-agent coverage applications.

B. Task Completion Time Comparisons

In multi-UAV coverage, completion time depends on the last UAV that finishes scanning. We assess load balancing across methods with 3 to 6 UAVs. To validate the effectiveness of the proposed balancing strategy, we first conducted an ablation study comparing Subgraph Init. (Algorithm 1) and Migration-only (Algorithm 2 with only node migration). Results demonstrate that while node migration provides coarse balancing, the subsequent node swapping mechanism effectively refines the partition, yielding a performance improvement of 0–11.6% over the migration-only phase. As shown in Table II, on average, the proposed method reduces completion time about 23.06% , 23.56% , 4.31% and 6.3% versus LSCPP, ACMC, DARP and the Gurobi solver, and has a slight increase within 1% versus TMSTC*. Fig. 6 illustrates the distribution of completion times. Our proposed method and TMSTC* demonstrate superior

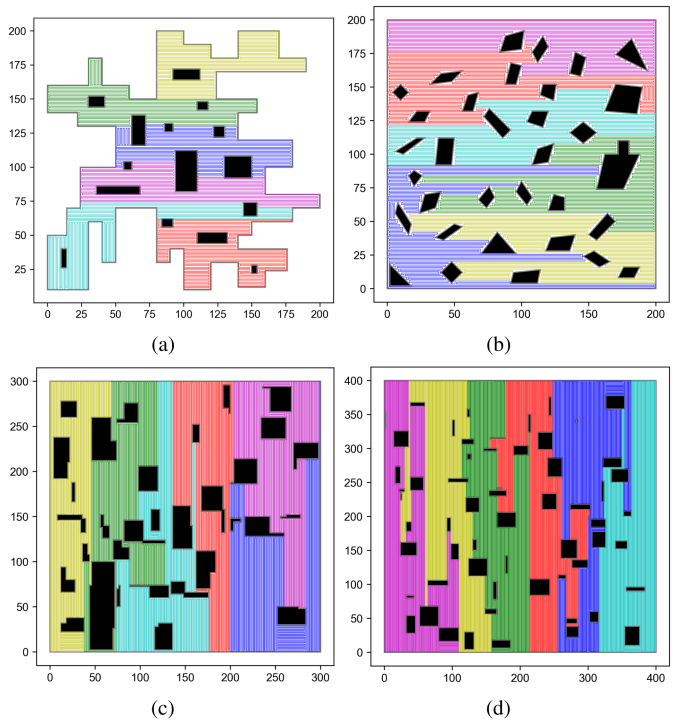

Fig. 5. Coverage Path Scenarios for 6 UAVs: (a) Case 1, (b) Case 2, (c) Case 3, (d) Case 4 using proposed method.

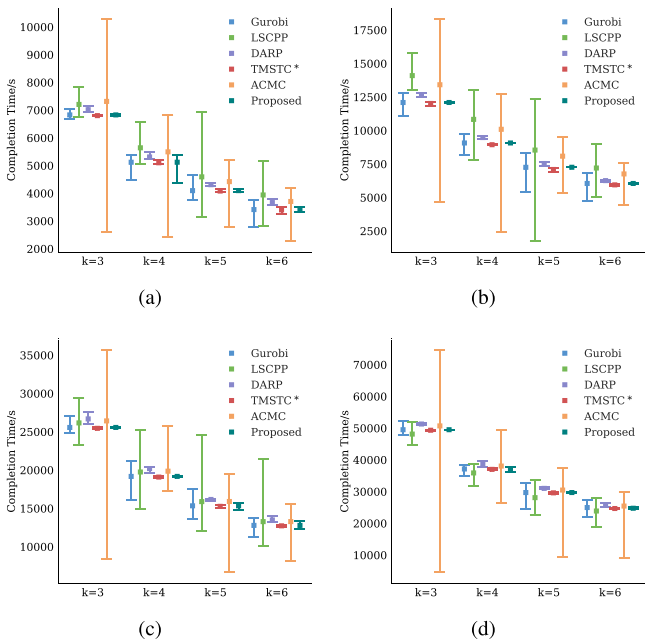

Fig. 6. The completion time for each method, under different scenarios and UAV numbers.

stability with average variations of 2.50% and 2.22% , respectively, outperforming the moderate fluctuation of DARP (3.68%). In sharp contrast, ACMC and LSCPP exhibit significant variability, exceeding 48% . Fig. 5(a)–(d) illustrates the planned paths in the 6-UAV scenario over areas from $200m \times 200m$ to $400m \times 400m$.

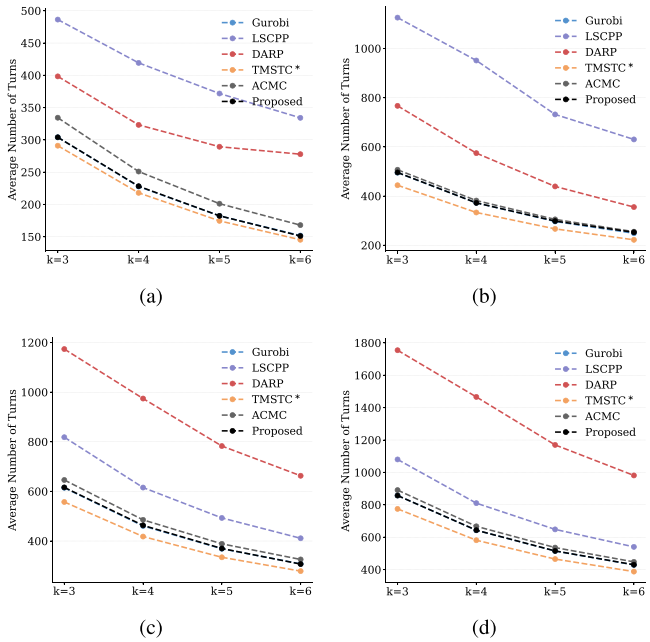

Fig. 7. The average number of turns for each method, under different scenarios and UAV numbers.

C. Turn Comparisons

As shown in Fig. 7, compared with LSCPP, DARP, the proposed method reduces the average number of turns by approximately 37.92% , 43.31% respectively. Additionally, it also reduces turns by 5.03% compared with ACMC. The proposed method's turn counts are close to those of TMSTC*, with an increase in turns within approximately 9.4% . Notably, the proposed and Gurobi-based methods both use a branch-connection strategy, yielding similar turn efficiency. By adapting branch directions to subregion shapes, the proposed method effectively reduces UAV turns and maintains efficiency across obstacle densities.

In summary, the proposed method significantly improves computational efficiency and offers better task load balancing compared to existing approaches.

VI. Conclusion

In this letter, we propose a hierarchical structure for the multi-UAV CPP problem, focusing on task load balancing and minimizing turning counts. The task assignment is addressed through a heuristic partitioning method with node migration and swapping, and multi-trees are constructed to ensure efficient coverage. The proposed method outperforms other approaches in large-scale scenarios with fast computation, making it suitable for real-time applications. Future work will extend this to dynamic environments by integrating real-time obstacle avoidance and adaptive velocity models.

References

[1] M. Höffmann, S. Patel, and C. Büskens, "Optimal guidance track generation for precision agriculture: A review of coverage path planning techniques," *J. Field Robot.*, vol. 41, no. 3, pp. 823–844, 2024.

[2] T. Kusnur and M. Likhachev, "Complete, decomposition-free coverage path planning," in *Proc. IEEE 18th Int. Conf. Automat. Sci. Eng.*, 2022, pp. 1431–1437.

[3] S. W. Cho, H. J. Park, H. Lee, D. H. Shim, and S.-Y. Kim, "Coverage path planning for multiple unmanned aerial vehicles in maritime search and rescue operations," *Comput. Ind. Eng.*, vol. 161, 2021, Art. no. 107612.

[4] H. Choset and P. Pignon, "Coverage path planning: The Boustrophedon cellular decomposition," in *Field and Service Robotics*. Berlin, Germany: Springer, 1998, pp. 203–209.

[5] K. Kumar and N. Kumar, "Region coverage-aware path planning for unmanned aerial vehicles: A systematic review," *Phys. Commun.*, vol. 59, 2023, Art. no. 102073.

[6] Y. Gabriely and E. Rimon, "Spanning-tree based coverage of continuous areas by a mobile robot," *Ann. Math. Artif. Intell.*, vol. 31, pp. 77–98, 2001.

[7] X. Huang, M. Sun, H. Zhou, and S. Liu, "A multi-robot coverage path planning algorithm for the environment with multiple land cover types," *IEEE Access*, vol. 8, pp. 198101–198117, 2020.

[8] J. Lu, B. Zeng, J. Tang, T. L. Lam, and J. Wen, "TMSTC*: A path planning algorithm for minimizing turns in multi-robot coverage," *IEEE Robot. Automat. Lett.*, vol. 8, no. 8, pp. 5275–5282, Aug. 2023.

[9] A. C. Kapoutsis, S. A. Chatzichristofis, and E. B. Kosmatopoulos, "DARP: Divide areas algorithm for optimal multi-robot coverage path planning," *J. Intell. Robotic Syst.*, vol. 86, pp. 663–680, 2017.

[10] X. Zheng, S. Koenig, D. Kempe, and S. Jain, "Multirobot forest coverage for weighted and unweighted terrain," *IEEE Trans. Robot.*, vol. 26, no. 6, pp. 1018–1031, Dec. 2010.

[11] N. Agmon, N. Hazon, G. A. Kaminka, and M. Group, "The giving tree: Constructing trees for efficient offline and online multi-robot coverage," *Ann. Math. Artif. Intell.*, vol. 52, no. 2, pp. 143–168, 2008.

[12] J. E. Beasley, "Route first—Cluster second methods for vehicle routing," *Omega*, vol. 11, no. 4, pp. 403–408, 1983.

[13] R. Nallusamy, K. Duraiswamy, R. Dhanalaksmi, and P. Parthiban, "Optimization of non-linear multiple traveling salesman problem using k-means clustering, shrink wrap algorithm and meta-heuristics," *Int. J. Nonlinear Sci.*, vol. 9, no. 2, pp. 171–177, 2010.

[14] I. Vandermeulen, R. Groß, and A. Kolling, "Balanced task allocation by partitioning the multiple traveling salesperson problem," in *Proc. Int. Conf. Auton. Agents Multiagent Syst.*, 2019, pp. 1479–1487.

[15] N. Byungsoo, "Heuristic approaches for no-depot K-traveling salesmen problem with a minmax objective," Ph.D. dissertation, Texas A&M University, College Station, TX, USA, 2006.

[16] S. Agarwal and S. Akella, "Area coverage with multiple capacity-constrained robots," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 3734–3741, Apr. 2022.

[17] J. Xie, L. R. G. Carrillo, and L. Jin, "An integrated traveling salesman and coverage path planning problem for unmanned aircraft systems," *IEEE Control Syst. Lett.*, vol. 3, no. 1, pp. 67–72, Jan. 2019.

[18] J. Xie and J. Chen, "Multiregional coverage path planning for multiple energy constrained UAVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 17366–17381, Oct. 2022.

[19] I. Vandermeulen, R. Groß, and A. Kolling, "Turn-minimizing multirobot coverage," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 1014–1020.

[20] S. Bochkarev and S. L. Smith, "On minimizing turns in robot coverage path planning," in *Proc. IEEE Int. Conf. Automat. Sci. Eng.*, 2016, pp. 1237–1242.

[21] E. M. Arkin, M. A. Bender, E. D. Demaine, S. P. Fekete, J. S. Mitchell, and S. Sethia, "Optimal covering tours with turn costs," *SIAM J. Comput.*, vol. 35, no. 3, pp. 531–566, 2005.

[22] W. H. Huang, "Optimal line-sweep-based decompositions for coverage algorithms," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2001, pp. 27–32.

[23] G. T. Toussaint, "Solving geometric problems with the rotating calipers," in *Proc. IEEE Mediterranean Electrotechnical Conf.*, 1983, Art. no. A10.

[24] J. Tang and H. Ma, "Large-scale multi-robot coverage path planning via local search," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 17567–17574.